ASSIGNMENT 1 REPORT

Title: Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System

Student: Santosh M.

Register Number: REPLACE_WITH_REGISTER_NUMBER

Course Outcome: CO1

Compiler used for verification: GCC g++ 14.2.0, C++17, Linux x86-64

Repository: DSA01-REGISTER_NUMBER-Assignment1 (replace placeholder with actual register number)

General design assumptions

• Store money as integer paise; Rs 1 = 100 paise.

• Peak hours are 08:00–09:59 and 17:00–19:59. Exactly 10:00 and 20:00 are off-peak.

• A successful stored-card trip must leave at least 5000 paise (Rs 50).

• The assignment does not state the student/senior concession percentage; it is not invented in Tasks 1–11.

• The weekend discount does not combine with the peak surcharge; weekend rule takes priority in Task 12.

Task 1 — Understand the problem and identify objects

Object-oriented analysis

• **SmartCard**: data: card ID and balance in integer paise; behaviour: recharge, check eligibility, deduct fare. The balance is private so callers cannot make it negative accidentally.

• **Passenger**: data: passenger category and concession eligibility; behaviour: request a journey. In a larger system, student/senior rules belong here or in a policy object.

• **Journey**: data: origin/destination zone, entry time, weekend flag; behaviour: validate inputs and provide journey facts to the fare calculator.

• **Zone**: a finite domain (`Inner`, `Middle`, `Outer`, later `Airport`) that selects a base fare.

• **FareTable/FarePolicy**: stores base fares and applies time-based surcharge or discounts. Centralising this rule avoids repeated magic numbers.

Procedural vs OOP

A procedural version keeps `balance`, fares and limits in variables and calls free functions such as `fareFor(zone, peak)` and `canTravel(balance, fare)`. It can be fast and compact, but every function must respect the same invariants. The OOP version groups state with operations and hides the balance, reducing accidental misuse and localising changes.

OOP principles

• **Encapsulation**: private card balance, modified only through validated methods.

• **Abstraction**: callers ask `canTravel()` / `deduct()` without knowing the internal balance representation.

• **Inheritance**: a `Passenger` base class could have `Student` and `SeniorCitizen` subclasses if their behaviour genuinely differs.

• **Polymorphism**: a virtual concession-policy method could calculate a concession per passenger type; for a small fixed rule, a data-driven policy may be simpler.

Invariants and assumptions

1. Money is stored as integer paise.

2. A successful journey leaves at least Rs 50 (5000 paise) after deduction.

3. An unsuccessful journey does not alter the balance.

4. Peak hours are `[08:00, 10:00)` and `[17:00, 20:00)`: 8:00 and 17:00 are peak; 10:00 and 20:00 are not.

5. Unknown zones, invalid times and negative recharges are rejected.

6. Since the brief does not specify concession percentages, Tasks 1–11 do not invent one; concessions are identified as a policy extension.

Alternative and complexity

Both a procedural function and a class method can compute one fare in O(1) time. A card balance occupies O(1) space in either design. OOP does not automatically make the algorithm faster; it makes invariants and rule changes easier to contain. A single fare change is ideally made in one central fare table/policy rather than in many duplicated conditionals.

Task 2 — Tokens, keywords, identifiers and constants

Examples of the five token groups: keywords (`constexpr`, `int`, `return`); identifiers (`BASE_PAISE`, `fare`); constants (`5000`, `20`); operators (`+`, `/`, `? :`); punctuators (`;`, `{}`, `()`, `[]`). `#define` performs textual substitution and has no C++ type or normal scope. `const` creates a non-modifiable object; `constexpr` permits compile-time evaluation when its initializer qualifies. `enum class Zone` scopes enumerators and prevents accidental implicit conversion to integers, unlike an unscoped `enum`. Use descriptive `UPPER_SNAKE_CASE` for constants and `lowerCamelCase` for functions/variables. A macro generally uses no storage itself; a constexpr value often compiles to an immediate constant, although storage may be needed if its address is taken. Runtime complexity is unchanged; type checking and scope safety are better with typed constants.`

Task 2 — Actual execution output

```
Fare paise: 3600
Minimum balance paise: 5000
```

Task 3 — Data types and compatibility

Value	Recommended type	Reason
Zone	`enum class Zone`	finite, type-safe set
Hour/minute	`int`	small validated integer range
Peak flag	`bool`	two states
Fare/balance	`long long` paise	exact arithmetic and large totals
Journey count	`long long`	comfortably handles millions and larger counts
Text/card ID	`std::string`	variable-length text
Approximate sensor values only	`double`	floating point is not appropriate for money

`sizeof` and limits are printed by `types.cpp` for the actual compiler/platform; sizes are implementation-dependent. On typical x86-64 GCC, `int` is 4 bytes and `long long`/`double` are 8 bytes. `INT_MAX` is typically 2,147,483,647. Three million journeys at Rs 36 total Rs 108,000,000,000 paise (Rs 108,000,000) if each fare is 3600 paise, which exceeds `int` in paise. `int + double` converts the integer to double; arithmetic with bool promotes it to int. `0.1` has no exact binary floating representation, so repeated additions may not equal the mathematically exact result. Integer paise avoids this for fares. Addition costs O(1) in either type; each `long long` and `double` is typically 8 bytes.

Task 3 — Actual execution output

```
sizeof(bool)=1
sizeof(char)=1
sizeof(int)=4
sizeof(unsigned)=4
sizeof(long long)=8
sizeof(float)=4
sizeof(double)=8
INT_MAX=2147483647 LLONG_MAX=9223372036854775807
3M journeys at Rs36 max: Rs 108000000
1M x 0.1 double = 100000.00000133288
1M x 10 paise integer = 10000000 paise
```

Task 4 — Prediction written before execution

Predicted output (before running):

```
3 3.5
C 68
1 true
s >= u
```

Explanation: `a / b` is integer division before conversion to double, so 7/2 becomes 3. `static_cast(a)` makes the division floating-point, giving 3.5. `'A' + 2` is integer arithmetic then converted to `char`, yielding `C`; `ch + 1` promotes the char to int and prints 68 (ASCII-compatible execution set). `bool flag = 5` converts nonzero to true; default bool output is `1`, and `boolalpha` prints `true`. In `s < u`, the negative signed `s` converts to unsigned, becoming a large unsigned value, so the condition is false and `s >= u` prints. Compiler/platform character encoding assumptions apply to the numeric character code.

Explicit conversions and brace initialisation make intent clearer and catch narrowing at compile time, but casts do not change the O(1) runtime or constant storage of these operations. Prefer validating signed inputs before converting to unsigned.

Actual output captured after execution:

```
3 3.5
C 68
1 true
s >= u
```

Task 5 — Prediction written before execution

Predicted output (before running):

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

Line 1: multiplication binds more tightly than addition: `5 + (3*2)=11`; parentheses force `(5+3)*2=16`. Line 2: postfix `x++` yields old 5 and then increments x; prefix `++y` increments y to 4 and yields 4; sum z=9, x=6, y=4. Line 3: conditional chooses max(6,4)=6; `%` and `*` have equal precedence and associate left-to-right: `(6%4)*2=4`. Line 4: integer division truncates toward zero; remainder has the dividend's sign: 10/3=3, 10%3=1, -10/3=-3, -10%3=-1. Line 5: i takes 0,3,6,9; k=18. `i = i++` is not a valid way to increment and can have undefined behaviour in older language versions / unsequenced-modification contexts; use `++i` or `i += 1`.

Peak check: `(hour >= 8 && hour < 10) || (hour >= 17 && hour < 20)`. `&&` binds tighter than `||`; parentheses improve readability. A 24-entry `bool` lookup table still checks in O(1) time but adds 24 bool elements (often 24 bytes); expression uses O(1) space too.

Actual output captured after execution:

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

Task 6 — Standard algorithmic pseudocode

```
ALGORITHM CalculateFare(zone, hour, weekend)
INPUT: valid zone, hour in 0..23, weekend flag
OUTPUT: fare in paise or INVALID
BEGIN
  SELECT zone
    Inner: base ← 1000
    Middle: base ← 2000
    Outer: base ← 3000
    OTHERWISE: RETURN INVALID
  END SELECT
  peak ← (hour >= 8 AND hour < 10) OR (hour >= 17 AND hour < 20)
  IF peak THEN fare ← base + (base * 20 DIV 100)
  ELSE fare ← base
  END IF
  RETURN fare
END

ALGORITHM ProcessJourney(cardBalance, fare)
BEGIN
  IF fare is INVALID OR cardBalance - fare < 5000 THEN
    OUTPUT "Denied"; keep balance unchanged
  ELSE
    cardBalance ← cardBalance - fare
    OUTPUT fare and cardBalance
  END IF
END
```

The completed C++ uses `switch` for the three fixed zones and integer paise. A `for` loop suits a known collection of trips; range-based `for` improves readability. `while` suits input until a sentinel or EOF. `continue` skips an invalid journey; `break` exits when no more input is required. A switch without a matching case and without `default` does nothing, so `default` is useful for invalid zones. Boundary policy: 10:00 and 20:00 are off-peak.

Task 6 — Actual execution output

```
Fare Rs 10.00 balance Rs 90
Fare Rs 36.00 balance Rs 54
Denied (low balance) balance Rs 54
```

Task 7 — Procedural vs OOP

Both versions are executable. In the procedural version, the balance is passed by reference and every caller must use `deduct` correctly. In the OOP version, `balance_` is private and can change only through `deduct`, which checks the invariant. The minimum balance is centralised in one constant, so a change is local. If the balance were public, any code could set it negative or bypass the minimum-balance rule. Both versions use O(1) time per check/deduction and O(1) state per card; encapsulation changes maintainability, not asymptotic complexity.

Task 7 — Actual execution output

```
Accepted, balance paise=7000
```

Task 8 — Safe input and output

`cin >>` parses formatted input. On letters where an integer is expected, `cin.fail()` becomes true. Call `cin.clear()` to reset the state, then `cin.ignore(max, '\\n')` to discard the invalid line before retrying. The example also validates the range 0–23. `setw(2)` and `setfill('0')` format paise; they affect the next formatted field, so fill is restored where appropriate. An alternative `getline` + `stoi` handles a whole line and exceptions, but costs O(L) for a line of length L and requires O(L) buffer space; formatted extraction is also O(L) in the consumed input and can recover with clear/ignore. `stoi` must additionally check that the entire line was consumed to reject trailing junk.

Task 8 — Actual execution output

```
Enter hour (0-23): Period: Peak
```

```
Fare: Rs 12.00
```

Task 9 — Implementation and test evidence

`fare_tests.cpp` table-tests off-peak fares, peak fares, and the exact 08:00, 10:00, 17:00 and 20:00 boundaries. Expected values are in paise. The input domain is validated for hours; a production CLI should parse zone text strictly and reject unknown zones, and recharge must reject negative values. A card whose balance is fare + 5000 paise is accepted; one paise less is denied. A suggested five-trip hand calculation at base fares: Inner off-peak Rs10 + Middle off-peak Rs20 + Outer off-peak Rs30 + Inner peak Rs12 + Middle peak Rs24 = **Rs96.00**. If using a card with Rs146 initial balance, the remaining balance is exactly Rs50 and all five trips can be accepted in order. A table-driven test loop takes O(t) time for t tests and O(t) space for the stored table; adding a boundary test is a one-row change.

Task 9 — Actual execution output

```
Inner off-peak: expected=1000 actual=1000 PASS
Inner peak: expected=1200 actual=1200 PASS
Middle off-peak: expected=2000 actual=2000 PASS
Middle peak: expected=2400 actual=2400 PASS
Outer off-peak: expected=3000 actual=3000 PASS
Outer peak: expected=3600 actual=3600 PASS
exact 08:00 peak: expected=1200 actual=1200 PASS
exact 10:00 off-peak: expected=1000 actual=1000 PASS
exact 17:00 peak: expected=1200 actual=1200 PASS
exact 20:00 off-peak: expected=1000 actual=1000 PASS
unknown zone: expected=-1 actual=-1 PASS
invalid hour: expected=-1 actual=-1 PASS
balance leaves exactly Rs50: PASS
balance leaves Rs49.99: PASS
negative recharge rejected: PASS
Five-trip hand total paise: 9600 (Rs 96.00)
Summary: 15/15 checks passed
```

Task 10 — Expectations vs execution

The program measures exact integer-paise totals and double totals for 20,000, 500,000 and 3,000,000 journeys at Rs12 each, and demonstrates a repeated floating-point addition. For integer multiplication, use `long long` before multiplication to avoid intermediate `int` overflow. Three million Rs36 fares total 10,800,000,000 paise, exceeding signed 32-bit `int` (2,147,483,647); the rupee total is Rs108,000,000. `long long` safely accommodates these values. `double` has a large range but repeated binary approximations can accumulate small errors. Addition is O(1) per item and each variable is O(1) space; typical `int` is 4 bytes, `long long` and `double` 8 bytes. Compiler warnings from `-Wall -Wextra` depend on code/compiler; do not claim warnings not actually observed. The included clean programs are compiled with those flags and warning logs are saved by the build script.

Task 10 — Actual execution output

```
20000 journeys: integer paise=24000000, rupees=240000.00
500000 journeys: integer paise=600000000, rupees=6000000.00
3000000 journeys: integer paise=3600000000, rupees=36000000.00
3M x Rs36 in paise=10800000000 (int max=2147483647)
3M x Rs0.10 accumulated double=300000.00000195682514459
```

Task 11 — Final recommendation

Retain the OOP SmartCard model, but modify the fare logic into a central `FarePolicy`/table rather than duplicating constants in switches. Use `enum class` for zones, `long long` integer paise for money and network totals, validated input, named `constexpr` policy constants, and private card balance. Fare changes then occur in one policy location and regression tests verify boundaries. Each journey is O(1); processing N journeys is O(N); storage per card is O(1). Limitations: hard-coded fares require recompilation/deployment, a fixed enum needs code changes for new zones, and real concession/pass eligibility needs an explicit business rule and persistent transaction/audit storage. At three million journeys/day, O(N) processing is practical in principle, but real throughput must be benchmarked on target hardware. A config file adds O(z) startup reading and O(z) fare storage for z zones, but allows policy updates without recompiling; it must validate malformed/missing values

and support safe versioning.

Task 11 — Actual execution output

```
Accepted fare=3600 paise, balance=6400 paise
```

Task 12 — Day-4 design challenge

A. Impact analysis

The fourth zone affects the `Zone` enum, fare table and zone validation. Weekend discount adds a named constant, a `weekend` input and a rule with priority over peak surcharge. Monthly passes require a new `MonthlyPass` class storing two covered zones and a coverage check. `SmartCard` remains useful for stored-value users; a journey authorisation layer should check the pass before charging stored value.

B. Evaluate the original design

A switch can be extended with `Airport`, but each new zone requires another branch and can miss a case. An indexed fare table makes adding a zone one row plus an enum update, but must validate enum/index values. Example: Airport base Rs60; weekday peak = Rs72 (7200 paise); weekend = Rs54 (5400 paise), because weekend discount does not combine with peak surcharge. A Rs100 card cannot pay Rs72 and retain Rs50, so stored-value travel is denied; a valid pass covering Airport can authorise the journey without fare deduction, subject to pass policy.

C. Adaptation

The working `revised_design.cpp` uses a four-entry `constexpr` fare table, integer paise, an independent weekend rule, and `MonthlyPass` covering two selected zones. For simplicity, this sample shows pass coverage but does not model expiry dates, purchase/refund or passenger identity. A production authorisation function should check pass coverage first, then apply stored-card eligibility and deduction only if the pass does not cover the zone.

D. Evidence-based defence

The included test prints Rs72.00 for Airport weekday peak and Rs54.00 for Airport weekend, and demonstrates that a pass can cover Inner and Airport but not Outer. The table is O(1) lookup and uses O(z) constant storage; the switch and table are both O(1) for a fixed zone set. Choose **MODIFY**, not full redesign: preserve SmartCard encapsulation and integer-paise invariants, while moving fare selection into a table and adding separate discount/pass policy components. Extend tests for all zones, all time boundaries, weekends, pass-covered/uncovered zones and card balance boundary cases before production release.

Task 12 — Actual execution output

```
Airport peak fare paise=7200 (Rs 72.00)
Airport weekend fare paise=5400 (Rs 54.00)
Pass covers Inner=1, Outer=0
Rs100 card can pay Rs72 and retain Rs50? 0
```

Overall complexity summary

Component	Time	Extra space
Fare lookup and peak check	O(1) per journey	O(1)
SmartCard eligibility/deduction	O(1)	O(1) per card
Processing N journeys	O(N)	O(1) streaming; O(N) if storing all journeys
Running t table-driven tests	O(t)	O(t) test table
Loading z fare entries from configuration	O(z) startup	O(z)

Build and validation commands

```
g++ -std=c++17 -Wall -Wextra -pedantic Task04/predict_output.cpp -o task04
./task04
bash scripts/build_and_test.sh
```

Submission checklist

- [ ] Replace `REPLACE_WITH_REGISTER_NUMBER` in README/report and rename repository to `DSA01--Assignment1`.
- [ ] Review and understand every program; rerun on the compiler used in college.
- [ ] Commit changes progressively (local history contains task-by-task commits after initialisation).
- [ ] Create/push the GitHub repository, confirm it is accessible to faculty, and add faculty collaborator if private.
- [ ] Submit repository URL and this report through Google Classroom.